

Week2

이나연

Part1

CS 224N Winter 2026 Assignment 2 Word2Vec and Dependency Parsing

Due date: January 22nd, Thursday, 4:30 PM PST

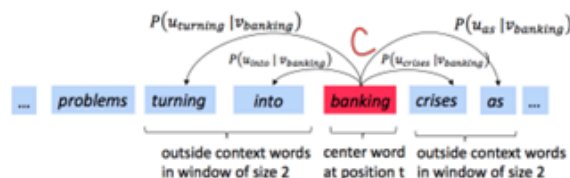
In this assignment, you will review the mathematics behind Word2Vec and build a neural dependency parser using PyTorch. For a review of the fundamentals of PyTorch, please check out the PyTorch review session on Canvas. In Part 1, you will explore the partial derivatives involved in training a Word2vec model using the naive softmax loss. In Part 2, you will learn about two general neural network techniques (Adam Optimization and Dropout). In Part 3, you will implement and train a dependency parser using the techniques from Part 2, before analyzing a few erroneous dependency parses.

If you are using LaTeX, you can use `\ifans{}` to type your solutions.

Please tag the questions correctly on Gradescope, otherwise the TAs will take points off if you don't tag questions.

1. Understanding word2vec (20 points)

Recall that the key insight behind word2vec is that 'a word is known by the company it keeps'. Concretely, consider a 'center' word c surrounded before and after by a context of a certain length. We term words in this contextual window 'outside words' (O). For example, in Figure 1 the context window length is 2, the center word c is 'banking', and the outside words are 'turning', 'into', 'crises', and 'as':



중심어 가져올 때
주변 단어 가 나타낼
조건부 확률
 $P(O=o | C=c)$

Figure 1: The word2vec skip-gram prediction model with window size 2

Skip-gram word2vec aims to learn the probability distribution $P(O|C)$. Specifically, given a specific word o and a specific word c , we want to predict $P(O=o|C=c)$: the probability that word o is an 'outside' word for c (i.e., that it falls within the contextual window of c). We model this probability by taking the softmax function over a series of vector dot-products:

$$P(O=o | C=c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \quad \text{softmax} \quad (1)$$

For each word, we learn vectors u and v , where $\mathbf{u}_o$ is the 'outside' vector representing outside word o , and $\mathbf{v}_c$ is the 'center' vector representing center word c . We store these parameters in two matrices, $\mathbf{U}$ and $\mathbf{V}$. The columns of $\mathbf{U}$ are all the 'outside' vectors $\mathbf{u}_w$; the columns of $\mathbf{V}$ are all of the 'center' vectors $\mathbf{v}_w$. Both $\mathbf{U}$ and $\mathbf{V}$ contain a vector for every $w \in \text{Vocabulary}$.¹

¹ Assume that every word in our vocabulary is matched to an integer number k . Bolded lowercase letters represent vectors. $\mathbf{u}_k$ is both the k^{th} column of $\mathbf{U}$ and the 'outside' word vector for the word indexed by k . $\mathbf{v}_k$ is both the k^{th} column of $\mathbf{V}$ and the 'center' word vector for the word indexed by k . In order to simplify notation we shall interchangeably use k to refer to word k and the index of word k .

Recall from lectures that, for a single pair of words c and o , the loss is given by:

$$\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U}) = -\log P(O = o | C = c). \quad (2)$$

We can view this loss as the cross-entropy between the true distribution $\mathbf{y}$ and the predicted distribution $\hat{\mathbf{y}}$, for a particular center word c and a particular outside word o . Here, both $\mathbf{y}$ and $\hat{\mathbf{y}}$ are vectors with length equal to the number of words in the vocabulary. Furthermore, the k^{th} entry in these vectors indicates the conditional probability of the k^{th} word being an 'outside word' for the given c . The true empirical distribution $\mathbf{y}$ is a one-hot vector with a 1 for the true outside word o , and 0 everywhere else, for this particular example of center word c and outside word o . The predicted distribution $\hat{\mathbf{y}}$ is the probability distribution $P(O | C = c)$ given by our model in equation (1).

Note: Throughout this homework, when computing derivatives, please use the method reviewed during the lecture (i.e. no Taylor Series Approximations).

- (a) (2 points) Prove that the naive-softmax loss (Equation 2) is the same as the cross-entropy loss between $\mathbf{y}$ and $\hat{\mathbf{y}}$, i.e. (note that $\mathbf{y}$ (true distribution), $\hat{\mathbf{y}}$ (predicted distribution) are vectors and $\hat{y}_o$ is a scalar):

$$\text{CE}(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_w y_w \log \hat{y}_w = -\log \hat{y}_o = -\log P(O=o|C=c) = -\sum_{w \in \text{Vocab}} y_w \log(\hat{y}_w) = -\log(\hat{y}_o). \quad (3)$$

* $\mathbf{y}$ is one-hot vector

Your answer should be one line. You may describe your answer in words.

- (b) (6 points) i. Compute the partial derivative of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to $\mathbf{v}_c$. Please write your answer in terms of $\mathbf{y}$, $\hat{\mathbf{y}}$, $\mathbf{U}$, and show your work to receive full credit.

softmax
①
$$\mathbf{J} = \frac{\exp(u_c^T \mathbf{v}_c)}{\sum_{w \in \text{V}} \exp(u_w^T \mathbf{v}_c)}$$

$$= -u_c^T \mathbf{v}_c + \log \sum_{w \in \text{V}} \exp(u_w^T \mathbf{v}_c)$$

$$\frac{\partial \mathbf{J}}{\partial \mathbf{v}_c} = -\mathbf{u}_c$$

$$+ \frac{1}{\sum_{w \in \text{V}} \exp(u_w^T \mathbf{v}_c)}$$

$$\times \sum_{x \in \text{V}} \exp(u_x^T \mathbf{v}_c) \mathbf{u}_x$$

$$= -\mathbf{u}_c + \sum_{x \in \text{V}} p(c) \mathbf{u}_x$$

$$= -\mathbf{u}_c + \mathbf{U} \mathbf{g}$$

$$= \mathbf{U}(\mathbf{g} - \mathbf{y})$$

↑
각 단어의
벡터 곱하기

- **Note:** Your final answers for the partial derivative should follow the shape convention: the partial derivative of any function $f(x)$ with respect to x should have the same shape as x .

- Please provide your answers for the partial derivative in vectorized form. For example, when we ask you to write your answers in terms of $\mathbf{y}$, $\hat{\mathbf{y}}$, and $\mathbf{U}$, you may not refer to specific elements of these terms in your final answer (such as y_1 , y_2 , ...). You may also not refer to specific vectors such as u_0 , u_1 , etc.

- ii. When is the gradient you computed equal to zero? Write a mathematical equation.

Hint: You may wish to review and use some introductory linear algebra concepts.

- iii. The gradient you found is the difference between the two terms. Provide an interpretation of how each of these terms improves the word vector when this gradient is subtracted from the word vector $\mathbf{v}_c$.

① $\mathbf{v}_c$ 가 $\mathbf{u}_c$ 에 가까워지도록 ② 오답의 벡터 끌어오도록

계산하면,
↓
 $\mathbf{v}_c$ 에 $\mathbf{u}_c$ 방향, $\mathbf{g}$ 방향이면 $\mathbf{v}_c$ 에서 더 크게 배움

- (c) (1 point) In many downstream applications using word embeddings, L2 normalized vectors (e.g. $\mathbf{u}/\|\mathbf{u}\|_2$ where $\|\mathbf{u}\|_2 = \sqrt{\sum_i u_i^2}$) are used instead of their raw forms (e.g. $\mathbf{u}$). Let's consider a hypothetical downstream task of binary classification of phrases as being positive or negative, where you decide the sign based on the sum of individual embeddings of the words. When would L2 normalization take away useful information for the downstream task? When would it not?

Hint: Consider the case where $\mathbf{u}_x = \alpha \mathbf{u}_y$ for some words $x \neq y$ and some scalar α . When α is positive, what will be the value of normalized $\mathbf{u}_x$ and normalized $\mathbf{u}_y$? How might $\mathbf{u}_x$ and $\mathbf{u}_y$ be related for such a normalization to affect or not affect the resulting classification?

²The cross-entropy loss between the true (discrete) probability distribution p and another distribution q is $-\sum_i p_i \log(q_i)$.

³Note that the true conditional probability distribution of context words for the entire training dataset would not be one-hot.

⁴This allows us to efficiently minimize a function using gradient descent without worrying about reshaping or dimension mismatching. While following the shape convention, we're guaranteed that $\theta := \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$ is a well-defined update rule.

L2 정규화
① 유용한 정보 없는 경우 : 단어의 강도 (intensity)나 빈도, 중요도 정보 사라질 때.
② 유용한 정보 잃지 않는 경우 : 단어의 크기 대신 방향이 결정적일 때, 정규화 후에도 벡터의 방향은 유지되므로 영향 없음.

(e)

CS 224n

Assignment 2

Page 3 of 8

$$J = -u_o^T v_c + \log \sum_{x \in V} \exp(u_o^T v_x)$$

$$\frac{\partial J}{\partial u_o} = -v_c + \frac{1}{\sum_{x \in V} \exp(u_o^T v_x)} \cdot \exp(u_o^T v_c) v_c$$

$$= -v_c + g_o v_c$$

$$= (g_o - 1) v_c$$

$$\frac{\partial J}{\partial u_w} = \frac{\partial}{\partial u_w} \log \sum_{x \in V} \exp(u_o^T v_x)$$

$$= \frac{1}{\sum_{x \in V} \exp(u_o^T v_x)} \cdot \exp(u_o^T v_c) v_c$$

$$= g_w v_c = (g_w - y_w) v_c$$

$$\therefore \frac{\partial J}{\partial u_w} = (g_w - y_w) v_c$$

$$\frac{\partial J}{\partial U} = \left[\frac{\partial J}{\partial u_1}, \frac{\partial J}{\partial u_2}, \dots, \frac{\partial J}{\partial u_{|V|}} \right]$$

(d) (1 point) Write down the partial derivative of $J_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to $\mathbf{U}$. Please break down your answer in terms of the column vectors $\frac{\partial J(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_1}, \frac{\partial J(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_2}, \dots, \frac{\partial J(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_{|Vocab|}}$ (do not further expand these terms). No derivations are necessary, just an answer in the form of a matrix.

(e) (5 points) Compute the partial derivatives of $J_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to each of the 'outside' word vectors, $\mathbf{u}_w$'s. There will be two cases: when $w = o$, the true 'outside' word vector, and $w \neq o$, for all other words. Please write your answer in terms of $\mathbf{y}$, $\hat{\mathbf{y}}$, and $\mathbf{v}_c$. In this subpart, you may use specific elements within these terms as well (such as $y_1, y_2, \dots$). Note that $\mathbf{u}_w$ is a vector while $y_1, y_2, \dots$ are scalars. Show your work to receive full credit.

$$\frac{df(y)}{dy} = \begin{cases} 1 & (y=1) \\ 0 & (y \neq 1) \end{cases}$$

(f) (2 points) As an additional exercise for this problem, you will be taking the derivatives of some common loss functions, which may be used in variations of word2vec (such as the negative sampling variant). The Leaky ReLU (Leaky Rectified Linear Unit) activation function is given by Equation 4 and Figure 2.

$$f(x) = \max(\alpha x, x) \quad (4)$$

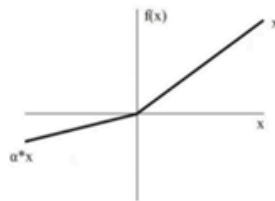


Figure 2: Leaky ReLU

Where x is a scalar and $0 < \alpha < 1$, please compute the derivative of $f(x)$ with respect to x . You may ignore the case where the derivative is not defined at 0.⁵

(g) (3 points) The sigmoid function is given by Equation 5.

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1} \quad (5)$$

Please compute the derivative of $\sigma(x)$ with respect to x , where x is a scalar. Please write your answer in terms of $\sigma(x)$. Show your work to receive full credit.

$$\frac{df(y)}{dy} = \sigma(y)(1 - \sigma(y))$$

$$\frac{df(y)}{dy} = \sigma(y)(1 - \sigma(y))$$

⁵If you're interested in how to handle the derivative at this point, you can read more about the notion of subderivatives.

Part2

(a) - (i) Momentum의 효과

- 이전 step에서의 파라미터 학습 방향을 어느정도 반영해 새 학습 방향을 정합니다. 업데이트의 분산이 줄어들고, 지속적으로 일관된 방향으로의 momentum이 붙어 더 빠르고 안정적으로 이동할 수 있습니다.

(a) - (ii) Adaptive Learning Rate 효과

- 그래디언트가 작거나 드물게 업데이트한 파라미터는 분모가 작아져 더 큰 업데이트가 가능하고, 기울기가 크고 빈번하게 업데이트된 파라미터는 분모가 커져 업데이트의 보폭이 줄어듭니다. 희소항 특징에 대응되는 파라미터도 충분히 학습될 수 있도록 하며, 전반적인 파라미터들의 학습 균형을 맞춰줍니다.

(b) - (i) Inverted Dropout의 γ 값 유도

- Dropout은 hidden layer의 일부 원소를 p_{drop} 의 확률에 따라 0으로 만들고, 살아남은 원소에 대해서는 γ 상수를 곱합니다. 살아남은 항들은 $(1 - p_{\text{drop}})$ 확률을 가지며, 아래와 같습니다.

$$\mathbb{E}[\mathbf{h}_{\text{drop}}]_i = \gamma \mathbb{E}[d_i] h_i = \gamma (1 - p_{\text{drop}}) h_i$$

따라서 $\gamma = 1/(1 - p_{\text{drop}})$ 입니다.

(b) - (ii) Dropout을 Train에는 적용하고, Evaluation에는 적용하지 않아야 하는 이유

- 훈련 시에는 뉴런을 무작위로 제거함으로써 특정 뉴런 간에 과도하게 의존하는 현상을 방지합니다.
- 또한 수많은 네트워크를 학습시키는 일종의 앙상블 효과를 통해 과적합을 방지합니다.
- 대신 평가 시에는 모든 뉴런과 파라미터를 온전히 활용하여, 매번 결과가 달라지지 않고 결정론적으로 안정된 최선의 예측 성능을 내야 하기 때문에 dropout을 적용하지 않습니다.

Part3

(a) parsing 표 채우기

Stack(처리중)	Buffer(대기열)	New dependency	Transition
[ROOT]	[I, presented, my, findings, at, the, NLP, conference]	[]	
[ROOT, I]	[presented, my, findings, at, the, NLP, conference]	[]	SHIFT

[ROOT, I, presented]	[my, findings, at, the, NLP, conference]	[]	SHIFT
[ROOT, presented]	[findings, at, the, NLP, conference]	(presented → I)	LEFT-ARC
[ROOT, presented, my]	[at, the, NLP, conference]	[]	SHIFT
[ROOT, presented, my, findings]	[at, the, NLP, conference]	[]	SHIFT
[ROOT, presented, findings]	[at, the, NLP, conference]	(findings → my)	LEFT-ARC
[ROOT, presented]	[at, the, NLP, conference]	(presented → findings)	RIGHT-ARC
[ROOT, presented, at]	[the, NLP, conference]	[]	SHIFT
[ROOT, presented, at, the]	[NLP, conference]	[]	SHIFT
[ROOT, presented, at, the, NLP]	[conference]	[]	SHIFT
[ROOT, presented, at, the, NLP, conference]	[]	[]	SHIFT
[ROOT, presented, at, the, conference]	[]	(conference → NLP)	LEFT-ARC
ROOT, presented, at, conference]	[]	(conference → the)	LEFT-ARC
[ROOT, presented, conference]	[]	(conference → at)	LEFT-ARC
[ROOT, presented]	[]	(presented → conference)	RIGHT-ARC

[ROOT]	[]	(ROOT → presented)	LEFT-ARC
--------	-----	--------------------	----------

(b) n개의 단어 가진 문장 파싱 몇 번 필요한지 n에 대해 나타내고 이유 설명하기

- 답 : 2n steps
- 설명 : 버퍼에 있는 n개의 단어가 스택으로 올라오기 위해서는 n번의 shift가 필요하고, 파싱이 완료된 후에는 스택에 있는 n개의 단어가 LEFT-ARC 또는 RIGHT-ARC를 통해 각각 한 번씩은 스택에서 제거되어야 하기 때문에, n번의 ARC 연산이 필요하므로 총 2n 단계가 필요합니다.

(c)

```
SHIFT test passed!
LEFT-ARC test passed!
RIGHT-ARC test passed!
parse test passed!
```

(d)

```
minibatch_parse test passed!
```

(e) - i

$$\frac{\partial h_i}{\partial x_j} = W_{ji} \cdot \mathbf{1}\{z_i > 0\}$$

(e) - ii

$$\frac{\partial J(\theta)}{\partial l_i} = -y_i + \frac{e^{l_i}}{\sum_{k=1}^3 e^{l_k}} = \hat{y}_i - y_i$$

*벡터 형태로 나타내면 $\frac{\partial J(\theta)}{\partial \mathbf{l}} = \hat{\mathbf{y}} - \mathbf{y}$

(e) - iii

```
Epoch 10 out of 10  
100%|███████████████████████████████████████████████████████████████| 1848/1848 [02:15<00:00, 13.68it/s]  
Average Train Loss: 0.06646781363141485  
Evaluating on dev set  
144585oit [00:00, 16088755.04it/s]  
- dev UAS: 88.73  
  
=====
```

```
TESTING  
=====
```

```
Restoring the best model weights found on the dev set  
Final evaluation on test set  
2919736it [00:00, 22833059.66it/s]  
- test UAS: 89.22  
Done!
```